Mobile Application Development

Week 10

Firebase Authentication

2025-26

Assistant Professor Taing PengSreng

Objectives

- Configure Firebase project for Flutter (Android/iOS)
- Initialize Firebase in Flutter
- Use FirebaseAuth methods: create, login, logout
- Handle auth state to auto-redirect between pages

What Firebase Auth Provides

- Fully managed identity system
- Supports: Email/Pass, Google, Apple, Phone, Anonymous
- Secure token generation & session tracking
- Built-in password reset + email verification

Firebase Setup (Flutter App)

CONTENTS

Steps

- Create project in Firebase Console
- Add Android App → download google-services.json
- Add iOS App → download GoogleService-Info.plist
- Add SDK packages:
  - flutter pub add firebase_core
  - flutter pub add firebase_auth

Initialize Firebase in main.dart

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(const MyApp());
}
```

- Must run before Firebase usage
- Required for all Firebase features

Auth State Listener (Root Navigation) [1]

- Use authStateChanges() to switch screens automatically:

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: StreamBuilder<User?>(
        stream: FirebaseAuth.instance.authStateChanges(),
```

Auth State Listener (Root Navigation) [2]

```
    builder: (_, snap) {
      if (snap.connectionState == ConnectionState.waiting)
        return const Center(child: CircularProgressIndicator());
      return snap.hasData ? const HomePage()
                          : const LoginPage();
    },
  ) ); }}
```

- Best practice: avoid manual navigation on login/logout
- Home/Login switches based on user presence

Create Account with Email & Password

```
await FirebaseAuth.instance.createUserWithEmailAndPassword(
  email: email.trim(),
  password: password,
);
```

- Errors include:
  - email-already-in-use
  - invalid-email
  - weak-password

Sign-In with Email & Password

```
await FirebaseAuth.instance.signInWithEmailAndPassword(
    email: email.trim(),
    password: password,
);
```

- If authentication succeeds → user object becomes non-null
- Auth stream triggers → UI shows HomePage

Sign-Out

```
await FirebaseAuth.instance.signOut();
```

- Auth stream resets to null → UI returns to LoginPage

CONTENTS

Login UI Requirements

- Email & password fields
- Validation (format + min length)
- Hide/show password
- Submit button → calls FirebaseAuth
- Show loading spinner
- Handle FirebaseAuth errors elegantly

Login Page (Trimmed Code) [1]

```
class LoginPage extends StatefulWidget {
  const LoginPage({super.key});
 @override State<LoginPage> createState() => _LoginPageState();
}
class _LoginPageState extends State<LoginPage> {
 final email = TextEditingController();
 final pass = TextEditingController();
 bool loading = false;
 String? error;
```

Login Page (Trimmed Code) [2]

```
Future<void> _login() async {
  setState(() { loading = true; error = null; });
try {
  await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: email.text.trim(),  password: pass.text );
} on FirebaseAuthException catch (e) {
  setState(() => error = e.message);
} finally {
  if (mounted) setState(() => loading = false);
}
}
```

CONTENTS

Login Page (Trimmed Code) [3]

```
@override
 Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(title: const Text("Login")),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            TextField(
              controller: _email,
              decoration:
                  const InputDecoration(labelText: "Email"),
            ),
```

Login Page (Trimmed Code) [4]

```
TextField(
  controller: _password,
  obscureText: true,
  decoration:
      const InputDecoration(labelText: "Password"),
),
if (_error != null)
  Text(
    _error!,
    style: const TextStyle(color: Colors.red),
  ),
const SizedBox(height: 12),
```

Login Page (Trimmed Code) [5]

```
            _loading  ? const CircularProgressIndicator()
              : ElevatedButton( onPressed: _login,  child: const Text("Login")  ),
            TextButton(
              onPressed: () => Navigator.push( context,
                MaterialPageRoute( builder: (_) => const SignUpPage()  ) ),
              child: const Text("Create an account"),
            ),
          ],
        )
      )
    );
}
```

Sign-Up Page (Trimmed Code)

```
await FirebaseAuth.instance.createUserWithEmailAndPassword(
 email: email.text.trim(),
 password: pass.text,
);
Navigator.pop(context); // Return to login
```

- Catch errors with FirebaseAuthException
- Suggest password ≥ 6 characters

Home Page (Sign-Out) [1]

```
class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) => Scaffold(
      appBar: AppBar(
        title: const Text("Home"),
        actions: [
```

Home Page (Sign-Out) [2]

```
IconButton(
    icon: const Icon(Icons.logout),
    onPressed: () => FirebaseAuth.instance.signOut() )
 ],
),
body: const Center(
  child: Text(  "Welcome! You are logged in.",
                    style: TextStyle(fontSize: 18)),
              ),
); }
```

CONTENTS

Handling Common FirebaseAuth Errors

- invalid-email → “Check email format”
- user-not-found → “No account with this email”
- wrong-password → “Incorrect password”
- email-already-in-use → “Try logging in instead”

Best Practices

- Always initialize Firebase at the top
- Use authStateChanges() for navigation
- Wrap calls in try/catch
- Show helpful messages